Mobile as a Platform: How Five Autonomous Teams Ship into One 53-Million-User App

Speaker: Ranjith Kumar Ragavan
Member of Technical Staff at T-Mobile

CONF42 PLATFORM ENGINEERING 2026



The Challenge

One Binary. Five Teams. Infinite Complexity.



Platform engineering is typically a cloud infrastructure conversation. But a large mobile app faces the same fundamental problem **inside a single binary**.

Without a platform layer, cross-module dependencies caused cascading build failures and release bottlenecks that stalled every team simultaneously.



53 Million Active Users

53M

Active Users

On the T-Life app

5

Engineering PODs

Fully autonomous teams

50+

Mobile Engineers

Android & iOS combined

99.9%

Crash-Free Rate

Sustained at scale

What Five Teams Actually Own

Each POD operates as an independent product team with full vertical ownership over its domain.

Account

Identity, profile management, authentication flows

Payments

Billing, autopay, one-time transactions

IoT

Connected device management and provisioning

Loyalty

T-Mobile Tuesdays, rewards, perks ecosystem

Core Shell

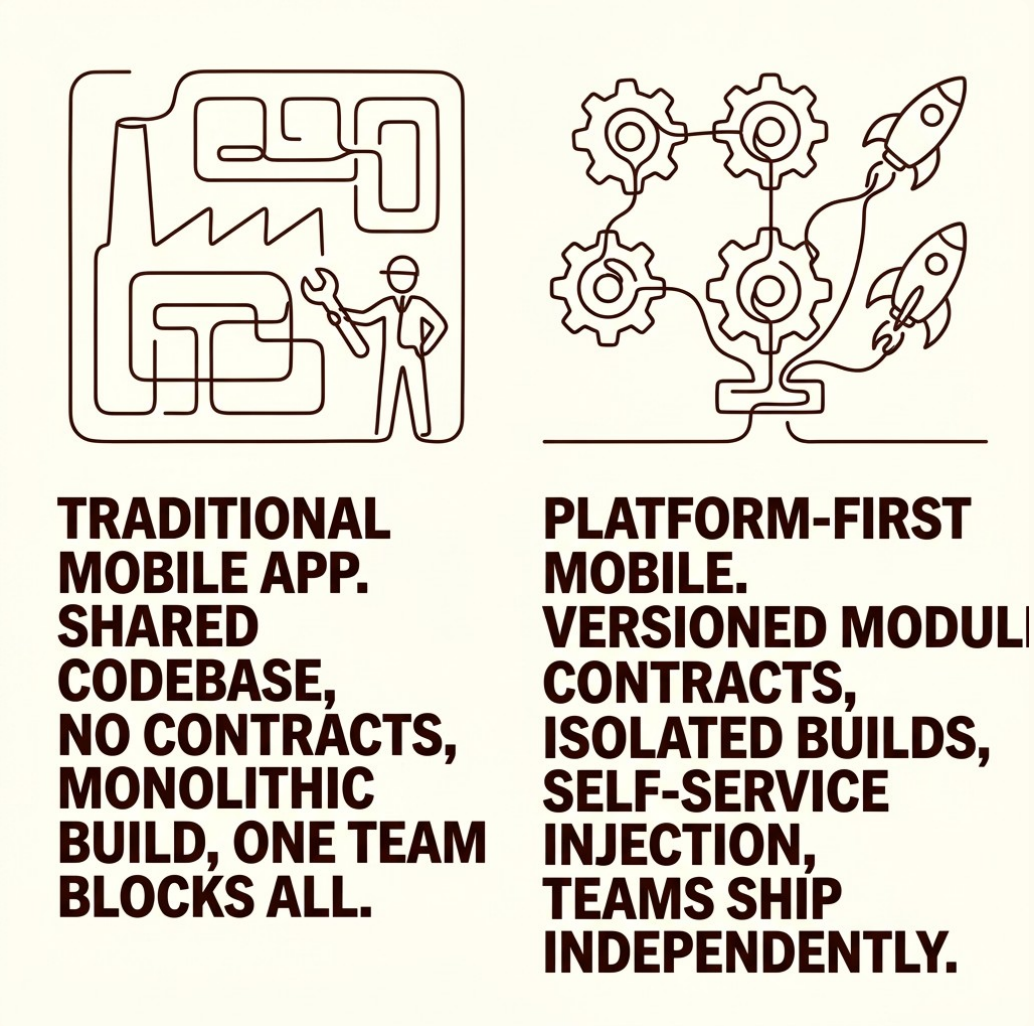
Navigation, DI, analytics, and shared infrastructure

Chapter 1

Treating the App Shell as an Internal Platform



The Core Insight: Platform Thinking for Mobile



The shift is conceptual before it is technical. The app shell stops being a shared codebase and becomes an **internal platform** with explicit guarantees just like a cloud platform exposes stable APIs to product teams.

Module teams become consumers of a platform, not contributors to a monolith.



Chapter 2

Versioned Interface Contracts

Contracts That Let Modules Evolve Independently

Each module exposes a typed interface its public API to the rest of the app. Changes are versioned. Consumers depend on the interface, never the implementation.

Typed Interfaces

Modules expose explicit public contracts. Internal implementation details are never shared across boundaries.

Semantic Versioning

Breaking changes are explicit and coordinated. Teams know exactly when a contract changes and why.

Consumer Independence

Teams can refactor or rewrite internals without notifying other teams — as long as the contract holds.

Chapter 3

Independent Build Pipelines

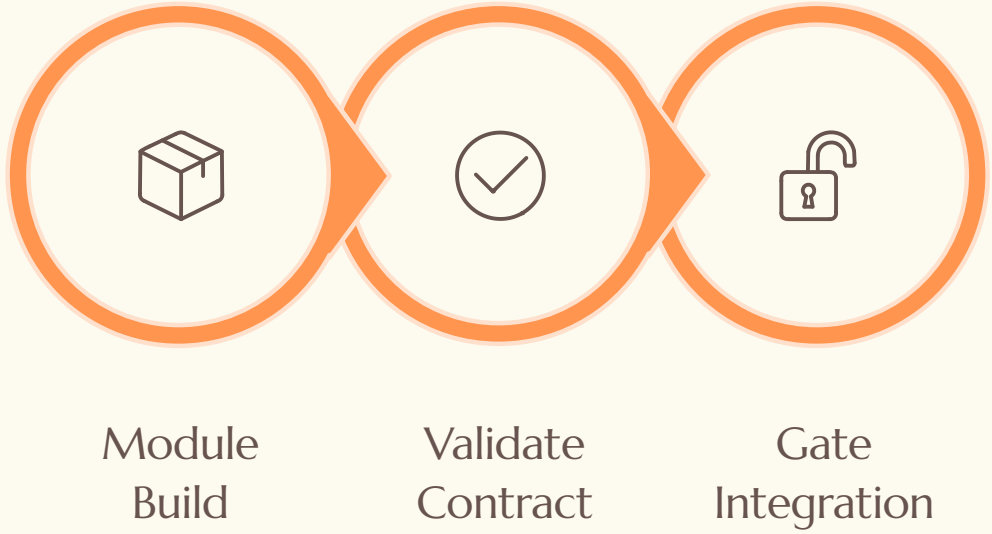


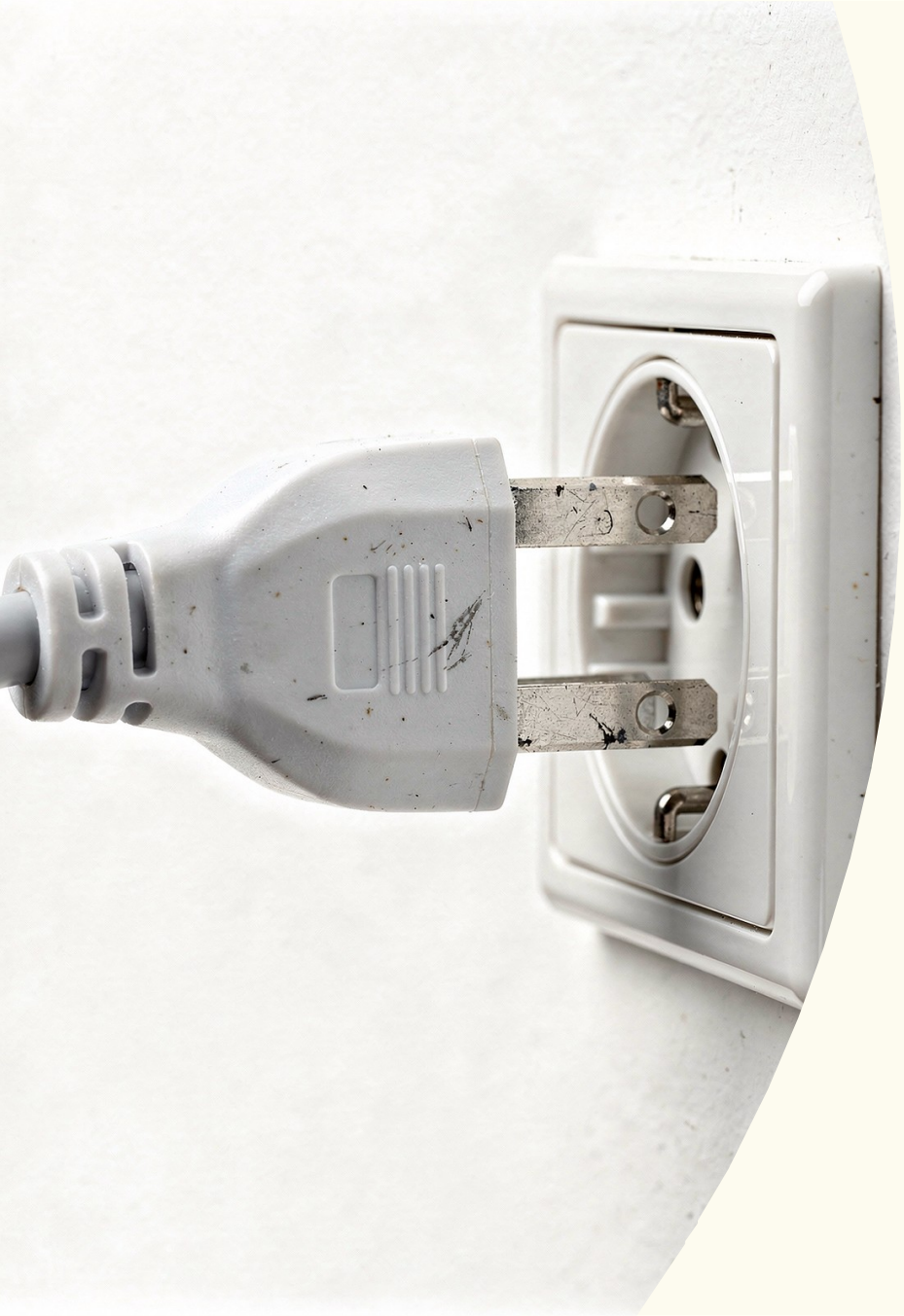
Isolating One Team's Failures from Everyone Else

In a monolithic build, a broken import in the payments module stalls the entire release. Independent pipelines eliminate that blast radius entirely.

Each module builds, tests, and validates against its own pipeline. Zero build regressions across teams is the measurable outcome not an aspiration.

Result: Zero cross-team build regressions sustained across all five PODs.





Chapter 4

Modular Feature Injection

Self-Service Shipping for Feature Teams

The feature-injection framework is the platform's most powerful self-service primitive. Teams register features at startup — no shell code changes required.

1

Register

Feature declares its entry points and dependencies via the injection registry

2

Inject

Shell discovers and mounts the feature at runtime without compile-time coupling

3

Ship

Team releases on their own cadence — no coordination with shell team needed

Chapter 5

Governance & State Patterns



Balancing Autonomy with Platform Stability

Governance Model

Platform stability requires guardrails without becoming a bottleneck. The shell team owns the contract but cannot block team velocity.

- RFC process for contract changes
- Automated lint rules enforce boundaries
- Shared ADR (Architecture Decision Record) log

Reactive State: MVI/MVVM

Tight coupling through shared state is as dangerous as tight coupling through imports. Reactive patterns enforce data consistency without module dependencies.

- Unidirectional data flow per module
- Events published to shared bus, never direct calls
- State scoped to module lifecycle

What You Take Back to Your Team

These are the patterns that delivered 99.9%+ crash-free sessions and zero cross-team build regressions at 53M-user scale.

01

Define your shell contract

Treat the app core as a platform with explicit, versioned APIs for every consuming team.

02

Isolate build pipelines

Each module builds independently. One team's red build never blocks another's green deploy.

03

Build a feature-injection registry

Give teams a self-service path to ship without touching shell code.

04

Adopt reactive state boundaries

MVI/MVVM patterns with scoped state prevent invisible coupling at runtime.

05

Govern through contracts, not control

RFCs and automated lint rules protect stability without slowing teams down.

Thank You!

Ranjith Kumar Ragavan
Member of Technical Staff at T-Mobile

Connect on LinkedIn
www.linkedin.com/in/about-ranjith

T-LIFE APP · T-MOBILE ENGINEERING

CONF42 PLATFORM ENGINEERING 2026

